

YOLOv11 Model Training

YOLOv11 Model Training

1. Course Content
2. Pre-trained Models
3. Training Traffic Sign Recognition Model (Case 1)
 - 3.1 Start Training
4. Training Lane Detection Model (Case 2)
5. Model Validation
 - 5.1 Test Traffic Sign Recognition Model
 - 5.2 Test Lane Detection Model
6. Source Code Analysis
 - 6.1 YOLOv11 Training Program
 - 6.2 Test Model Program

[!IMPORTANT]

- The factory image already contains pre-trained traffic sign recognition models. This section tutorial is provided for users who need to train YOLOv11 object detection models themselves and master model training methods.
- If you want to experience the complete functionality directly, you can skip this section!!!
- If using the dataset in the factory image to train the model, you need to unzip the .zip format dataset compressed package first.

1. Course Content

1. Master the method for training YOLOv11 object detection models.

2. Pre-trained Models

- The factory image already contains pre-trained traffic sign recognition models and lane detection models. The paths are as follows:
- yolo11n.pt base model

```
/home/jetson/MODELS/yolov11/yolo11n.pt
```

```
jetson@yahboom:~/MODELS/yolov11$ pwd
/home/jetson/MODELS/yolov11
jetson@yahboom:~/MODELS/yolov11$
```

- Lane detection model path:

```
/home/jetson/MODELS/auto_drive/lane_v6_int8.engine
```

- Traffic sign recognition model path:

```
/home/jetson/MODELS/auto_drive/sign_model.engine
```

```
jetson@yahboom:~/MODELS/auto_drive$ ls  
lane_V6_int8.engine  sign_model.engine  
jetson@yahboom:~/MODELS/auto_drive$
```

3. Training Traffic Sign Recognition Model (Case 1)

The program path for training YOLOv11 object detection models:

```
/home/jetson/yahboomM3_ws/yolo_train/train.py
```

Optional startup parameter descriptions:

- **project:** Training output folder, default is `./project`
- **MODEL_PATH:** YOLOv11 base model path, default is `/home/jetson/MODELS/yolov11/yolo11n.pt`
- **epochs:** Training rounds, the more rounds the better the recognition model effect, default is 100 rounds
- **imgsz:** Training image size, default is 640
- **train_config:** Configuration file path, used to describe dataset path and target categories

3.1 Start Training

- Here we use 50 training rounds as an example (it is recommended to train 100 rounds or more):

```
cd ~/yahboomM3_ws/yolo_train
```

```
python3 train.py --epochs 50 --  
train_config=/home/jetson/yahboomM3_ws/yolo_train/train_sign/sign.yaml
```

```

jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom:~/yahboomM3_ws/yolo_train 147x40
jetson@yahboom:~/yahboomM3_ws/yolo_train$ python3 train.py --epochs 50 --train_config=/home/jetson/yahboomM3_ws/yolo_train/train_sign/sign.yaml
New https://pypi.org/project/ultralytics/8.3.228 available 🐼 Update with 'pip install -U ultralytics'
ultralytics 8.3.214 Python-3.10.12 torch-2.5.0a0+872d972e41.nv24.08 CUDA:0 (Orin, 7620MiB)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=andaugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, classes=N
one, close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=/home/jetson/yahboomM
3_ws/yolo_train/train_sign/sign.yaml, deterministic=True, device=None, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None, epo
chs=50, erasing=0.4, exist_ok=False, flipplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv
_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, lline_width=None, lr0=0.01, lr=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train
n, model=/home/jetson/MODELS/yolov11/yolo11n.pt, momentum=0.937, mosaic=1.0, multi_scale=False, name=train2, nbs=64, nms=False, opset=None, optimiz
e=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=/project/
train2, rect=False, resume=False, retina_masks=False, save=True, save_conf=False, save_crop=False, save_dir=/home/jetson/yahboomM3_ws/yolo_train/project/
train2, save_frames=False, save_json=False, save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=Tr
ue, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=None, tracker=botsort.yaml, t
ranslate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.000
5, workers=8, workspace=None
COMET ERROR: Failed to create Comet experiment, reason: ValueError('Comet.ml requires an API key. Please provide as the first argument to Experimen
t(api_key) or as an environment variable named COMET_API_KEY ')
WARNING ⚠ Comet installed but not initialized correctly, not logging this run. Comet.ml requires an API key. Please provide as the first argument t
o Experiment(api_key) or as an environment variable named COMET_API_KEY
Overriding model.yaml nc=80 with nc=14

      from  n  params  module  arguments
0         -1  1      464  ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1         -1  1     4672  ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2         -1  1     6640  ultralytics.nn.modules.block.C3k2  [32, 64, 1, False, 0.25]
3         -1  1    36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
4         -1  1   260800  ultralytics.nn.modules.block.C3k2  [64, 128, 1, False, 0.25]
5         -1  1   147712  ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
6         -1  1    87040  ultralytics.nn.modules.block.C3k2  [128, 128, 1, True]
7         -1  1   295424  ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]
8         -1  1   346112  ultralytics.nn.modules.block.C3k2  [256, 256, 1, True]
9         -1  1   164608  ultralytics.nn.modules.block.SPPF  [256, 256, 5]
10        -1  1   249728  ultralytics.nn.modules.block.C2PSA  [256, 256, 1]
11        -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
12        [-1, 6] 1         0  ultralytics.nn.modules.conv.Concat  [1]
13        -1  1   111296  ultralytics.nn.modules.block.C3k2  [384, 128, 1, False]
14        -1  1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
15        [-1, 4] 1         0  ultralytics.nn.modules.conv.Concat  [1]
16        -1  1   32096  ultralytics.nn.modules.block.C3k2  [256, 64, 1, False]
17        -1  1   36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
18        [-1, 12] 1         0  ultralytics.nn.modules.conv.Concat  [1]

```

- During the training process, the training duration and loss values for each round will be displayed

```

jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom:~/yahboomM3_ws/yolo_train 151x45
train: Fast Image access (ping: 0.0±0.0 ms, read: 87.2±37.6 MB/s, size: 36.6 KB)
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 88 images, 1 backgrounds, 0 corrupt: 28%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 180 images, 4 backgrounds, 0 corrupt: 58%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 283 images, 4 backgrounds, 0 corrupt: 91%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
train: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
train: New cache created: /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/train/labels.cache
val: Fast Image access (ping: 0.0±0.0 ms, read: 104.1±24.4 MB/s, size: 44.4 KB)
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 88 images, 1 backgrounds, 0 corrupt: 28%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 166 images, 4 backgrounds, 0 corrupt: 53%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 298 images, 4 backgrounds, 0 corrupt: 96%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
val: Scanning /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels... 311 images, 4 backgrounds, 0 corrupt: 100%
val: New cache created: /home/jetson/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset/val/labels.cache
WARNING ⚠ Unable to import Axes3D. This may be due to multiple versions of Matplotlib being installed (e.g. as a system package and as a pip package). As a result, the 3D projection is not available.
Plotting labels to /home/jetson/yahboomM3_ws/yolo_train/project/train2/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...
optimizer: AdamW(lr=0.000556, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 6 dataloader workers
Logging results to /home/jetson/yahboomM3_ws/yolo_train/project/train2
Starting training for 50 epochs...

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/50    2.2G      1.417    4.511    1.26      46         640: 100% 20/20 0.91t/s 22.4s
Class   Images  Instances  Box(P   R
all      311      724      0.0569 0.0359 0.0456 0.0407 10/10 1.91t/s 5.3s

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
2/50    2.31G     1.131    3.525    1.041     18         640: 100% 20/20 1.61t/s 12.2s
Class   Images  Instances  Box(P   R
all      311      724      0.0298 0.546  0.163  0.113 10/10 1.61t/s 6.1s

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
3/50    2.31G     1.167    2.58     1.039     31         640: 100% 20/20 1.41t/s 14.2s
Class   Images  Instances  Box(P   R
all      311      724      0.025  0.188  0.271  0.191 10/10 1.41t/s 7.1s

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
4/50    2.31G     1.171    2.074    1.05      20         640: 100% 20/20 1.41t/s 14.2s
Class   Images  Instances  Box(P   R
all      311      724      0.025  0.188  0.271  0.191 7/10 0.91t/s 7.0s<3.2s

```

- **Epoch:** Current training round
- **GPU_mem:** GPU memory currently occupied by training
- **box_loss:** Bounding box regression loss value, measuring the difference between predicted boxes and ground truth boxes (position, size). The smaller the value, the more accurate the box positioning.
- **cls_loss:** Classification loss value, measuring the difference between predicted classes and ground truth classes. The smaller the value, the more accurate the classification.
- **dfl_loss:** Distribution focal loss, used to optimize bounding box regression accuracy, especially for small targets. The smaller the value, the more accurate the bounding box detail prediction.
- **Instances 46:** Total number of target instances contained in the current batch.

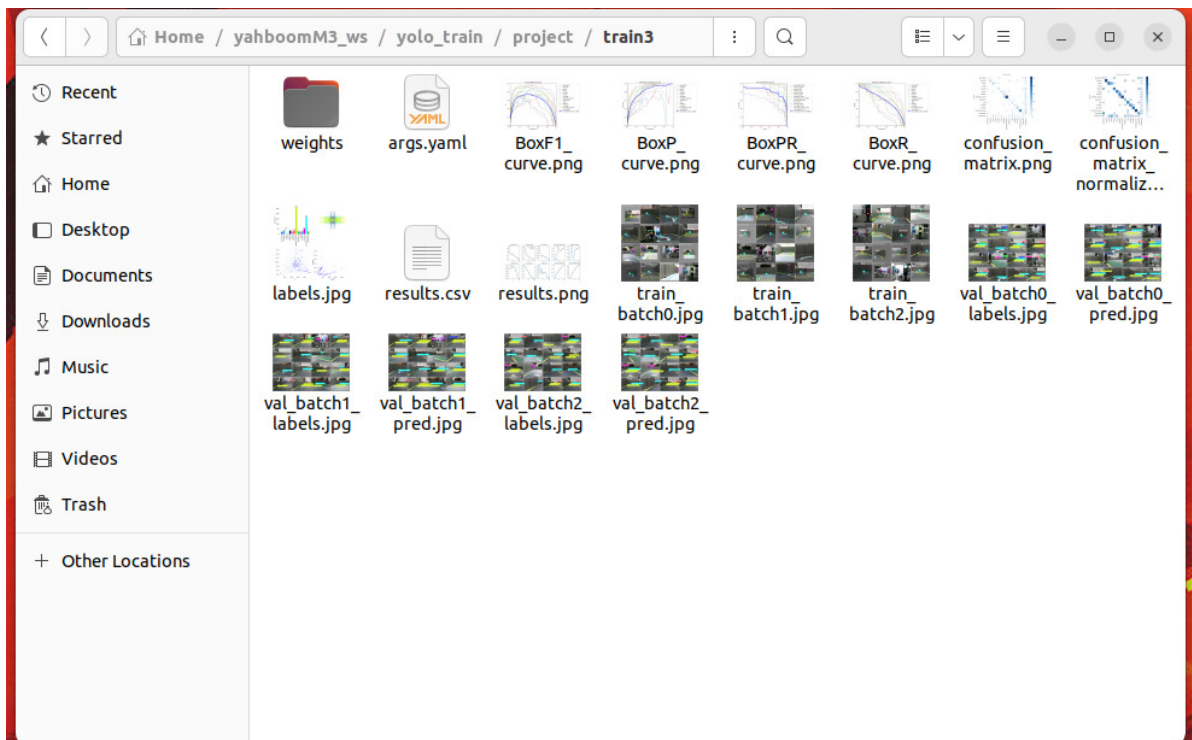
- **Size 640:** Input image size is 640x640 (YOLOv11 default training input size).
- After training is completed, the terminal will prompt the save path of the training output files

```

jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom: ~/yahboomM3_ws/yolo_train 110x33
Validating /home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.pt...
Ultralytics 8.3.214 Python-3.10.12 torch-2.5.0a0+872d972e41.nv24.08 CUDA:0 (Orin, 7620MiB)
YOLO11n summary (fused): 100 layers, 2,584,882 parameters, 0 gradients, 6.3 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 10% 1/1
0 0.5it/s 0.7s<19. Class Images Instances Box(P R mAP50 mAP50-95): 20
% 2/10 0.7it/s 1.5s<11. Class Images Instances Box(P R mAP50 mAP50-95): 30%
P50 mAP50-95): 40% 3/10 0.6it/s 4.4s<12. Class Images Instances Box(P
R mAP50 mAP50-95): 50% 4/10 0.7it/s 5.6s<9.1 Class Images Ins
tances Box(P R mAP50 mAP50-95): 60% 5/10 0.9it/s 6.2s<5.3 Cla
ss Images Instances Box(P R mAP50 mAP50-95): 70% 6/10 1.1it/s 6.9s<3.5
Class Images Instances Box(P R mAP50 mAP50-95): 80% 7/10
1.4it/s 7.4s<2.2 Class Images Instances Box(P R mAP50 mAP50-95): 90%
8/10 1.4it/s 8.0s<1.4 Class Images Instances Box(P R mAP50 mAP50-95): 100%
9/10 1.6it/s 8.6s<0.6 Class Images Instances Box(P R mAP50 mAP50-95): 10/10 1.1it/s 9.0s
all 311 724 0.859 0.739 0.817 0.654
greenlight 16 16 0.626 0.525 0.594 0.483
honking 29 29 0.862 1 0.99 0.825
parkA 50 50 0.914 0.637 0.905 0.67
parkB 2 2 1 0 0.0167 0.0117
redlight 30 31 0.754 0.903 0.868 0.68
school 19 19 0.965 0.474 0.82 0.656
sidewalk 34 34 0.918 1 0.995 0.848
sidewalk_ground 165 261 0.965 0.981 0.983 0.636
speedlimit 20 20 0.847 0.6 0.828 0.715
stop 14 14 0.954 0.929 0.974 0.843
straight 16 16 0.871 0.812 0.867 0.759
turnright 51 51 0.977 0.848 0.955 0.825
turnright_ground 164 165 0.91 0.915 0.973 0.664
yellowlight 16 16 0.468 0.715 0.67 0.542
Speed: 0.9ms preprocess, 43.3ms inference, 0.0ms loss, 5.5ms postprocess per image
Results saved to /home/jetson/yahboomM3_ws/yolo_train/project/train3
jetson@yahboom: ~/yahboomM3_ws/yolo_train$

```

- In the output folder we can see the following contents
- **weights:** Trained model weight files
- **.png:** Score data during training process



4. Training Lane Detection Model (Case 2)

- The training method is the same as before

```
cd ~/yahboomM3_ws/yolo_train
```

- Start training

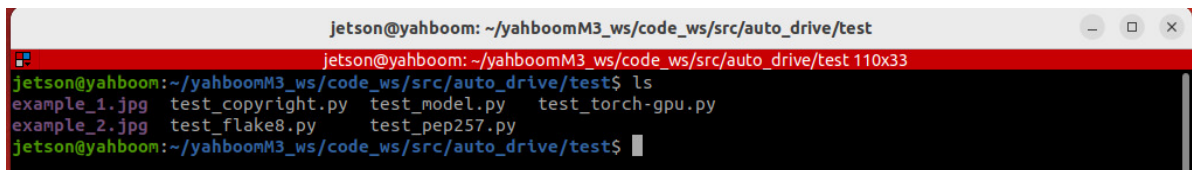
```
python3 train.py --epochs 50 --  
train_config=/home/jetson/yahboomM3_ws/yolo_train/train_lane/lane.yaml --project  
'./project2'
```

Here the `--project` parameter specifies the folder for training output files, used to distinguish from the previous traffic sign recognition model. The path is set to `'./project2'`. After training is completed, the model weight files will be saved in the path specified by the project parameter.

5. Model Validation

The program for testing the model and example images are located in the test folder of the `auto_drive` package. Use the following command to view the relevant content:

```
cd ~/yahboomM3_ws/code_ws/src/auto_drive/test  
ls
```



A terminal window screenshot showing the command `ls` being executed in the directory `~/yahboomM3_ws/code_ws/src/auto_drive/test`. The output lists the following files: `example_1.jpg`, `test_copyright.py`, `test_model.py`, `test_torch-gpu.py`, `example_2.jpg`, `test_flake8.py`, and `test_pep257.py`.

[!NOTE]

The test folder in the `auto_drive` package contains test programs and test images for testing model effects, used to verify whether the trained model meets expectations. Among them, `example_1.jpg` is a lane image for testing the lane detection model; `example_2.jpg` is an image containing traffic signs for testing the traffic sign recognition model; `test_model.py` is the test program.

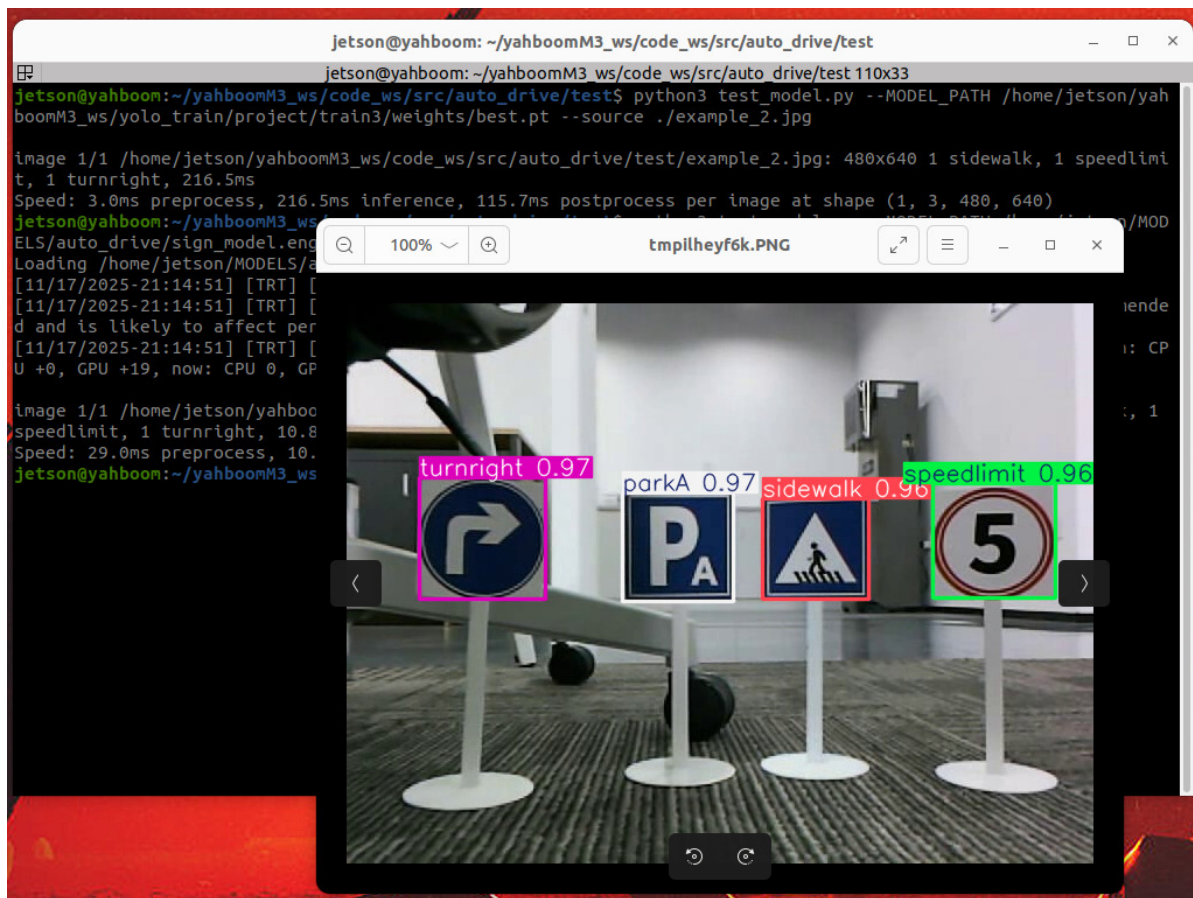
Optional startup parameters for `test_model.py`:

- **MODEL_PATH**: Path to the model weight file to be tested
- **half**: Whether to perform inference in FP16 half-precision mode to shorten inference time, default is `True`
- **device**: Device for model inference, default is `cuda:0`
- **source**: Image to be tested

5.1 Test Traffic Sign Recognition Model

- Replace `MODEL_PATH` with the actual model weight path output during training
- Select `./example_2.jpg` for source


```
python3 test_model.py --MODEL_PATH
/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.pt --source
./example_2.jpg
```

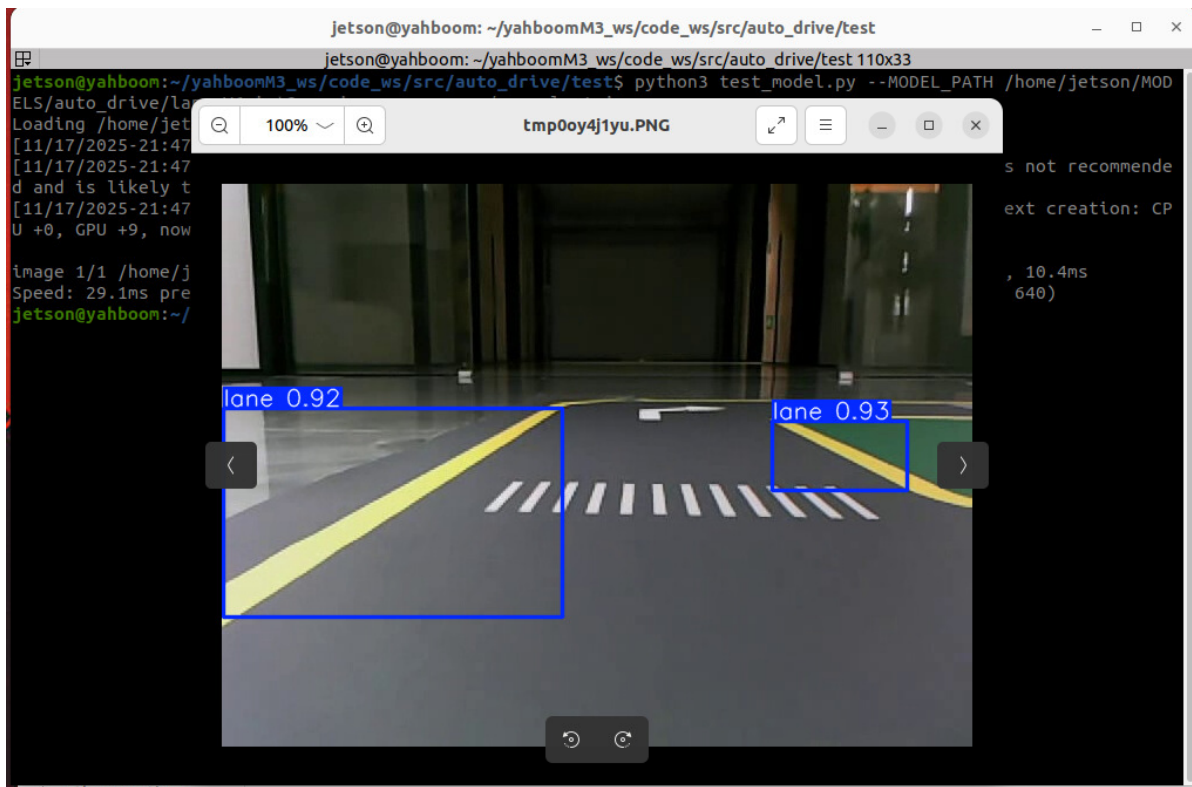


5.2 Test Lane Detection Model

- Replace MODEL_PATH with the actual model weight path output during training
- Select ./example_1.jpg for source

```
python3 test_model.py --MODEL_PATH
/home/jetson/yahboomM3_ws/yolo_train/project2/train3/weights/best.pt --source
./example_1.jpg
```

- You can see that it correctly recognizes the left and right lane lines and displays the confidence of the detection boxes



6. Source Code Analysis

6.1 YOLOv11 Training Program

The program source code is located at `/home/jetson/yahboomM3_ws/yolo_train/train.py`

- Configuration parameters required during training are defined through the `argparse` library, allowing parameters to be passed through the command line when running the script
- During the training process, logs, loss curves, evaluation metrics (such as mAP), and model weight files (best.pt, last.pt, etc.) are automatically generated and saved in the directory specified by `--project`.

```
import argparse
from ultralytics import YOLO

def get_args():
    """
    """

    parser = argparse.ArgumentParser()
    parser.add_argument('--project', default='./project', help='Project file
output directory', type=str)
    parser.add_argument('--
MODEL_PATH', default='/home/jetson/MODELS/yolov11/yolo11n.pt', help='原始模型权重文
件路径Original model weight file path', type=str)
    parser.add_argument('--epochs', default=100, help='训练轮次Training rounds',
type=int)
    parser.add_argument('--imgsz', default=640, help='图像大小Image size',
type=int)
    parser.add_argument('--train_config', default='', help='配置文件路径
Configuration file path', type=str)
    return parser.parse_args()
```

```

if __name__ == "__main__":
    args = get_args()
    model = YOLO(args.MODEL_PATH) # Load a pretrained model

    results = model.train(data=args.train_config, epochs=args.epochs,
imgsz=args.imgsz,project=args.project)#Start training

```

6.2 Test Model Program

The program source code is located at

`/home/jetson/yahboomM3_ws/code_ws/src/auto_drive/test/test_model.py`

- Configuration parameters required for testing the model are defined through the `argparse` library, allowing parameters to be passed through the command line when running the script
- `half=True`: Enable FP16 half-precision inference, which can significantly improve speed on GPU
- `result.show()`: Is a convenient method provided by the Ultralytics library that directly displays the image annotated with detection boxes, classes, and confidence in a window

```

from ultralytics import YOLO
import argparse
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--MODEL_PATH', default='', help='模型文件路径', type=str)
    parser.add_argument('--half', default=True, help='fp16精度FP16 precision',
type=str)
    parser.add_argument('--device', default="cuda:0", help='推理设备Reasoning
device', type=str)
    parser.add_argument('--source', default="./example_1.jpg", help='测试图片Test
images', type=str)
    return parser.parse_args()

if __name__ == "__main__":
    args = get_args()
    yolo_model = YOLO(args.MODEL_PATH, task="detect")
    results = yolo_model(args.source, device=args.device, half=args.half)
    for result in results:
        boxes = result.boxes
        result.show()

```